

1. Introduction

- 순환 신경망 (RNN) 및 그 변형들(LSTM, GRNN)은 언어 모델링과 기계 번역 문제에서 주로 사용되며, 시퀀스 모델링에 적합한 방법으로 자리잡음. 하지만 계산의 순차적 특성 때문에 훈련에서 병렬 처리가 어려움.
- attention 메커니즘은 입력과 출력 시퀀스 간의 관계를 모델링하면서도 시퀀스의 거리를 무시하고 의존성을 처리할 수 있게 함. 대개는 순환 신경망과 함께 사용됨.
- 트랜스포머는 순환 신경망을 사용하지 않고, 대신 attention 메커니즘만으로 입력과 출력 간의 의존성을 처리함. 이 모델은 병렬 처리가 가능하고, 훈련 시간이 짧은데도 번역 품질에서 새로운 최첨단 성능을 달성할 수 있음.

2. Background

- 확장된 신경망 GPU, ByteNet, ConvS2S는 합성곱 신경망을 사용하여 모든 입력과 출력 위치에서 병렬로 계산을 처리하지만, 멀리 떨어진 위치들 간의 의존성을 학습하는 데 어려움이 있음.
- 트랜스포머는 입력과 출력의 의존성을 관계짓는 데 있어 고정된 연산 횟수를 사용하여 이 문제를 해결. 다만, 평균화된 attention 가중치로 인한 해상도 감소가 발생하고, 이를 Multi-Head Attention으로 보완.
- Self-attention은 단일 시퀀스의 위치들 간의 관계를 모델링하며, 여러 작업에서 유용하게 사용됨.
- End-to-end 메모리 네트워크는 순환 대신 순차적 attention 메커니즘을 사용하여 질문 응답 및 언어 모델링에서 좋은 성과를 낸 사례가 있음.
- 트랜스포머는 self-attention에 전적으로 의존하는 첫 번째 모델로, RNN이나 합성곱을 사용하지 않음.

3. Model Architecture

- 인코더-디코더 구조: 트랜스포머는 멀티 헤드 attention과 완전 연결 피드포워드 네트워크를 사용하여 인코더와 디코더를 구성. 각 레이어는 잔차 연결과 레이어 정규화를 사용.
- Attention 메커니즘: Attention 함수는 쿼리와 키-값 쌍을 통해 출력을 생성. "Scaled Dot-Product Attention"은 내적 계산 후 스케일링하여 가중치를 부여하고, 멀티 헤드 attention은 다양한 표현 하위공간을 동시에 처리.
- Self-attention: 인코더와 디코더 모두에서 self-attention을 사용하여 각 위치가 다른 위치들을 참조하도록 함.
- 위치별 피드포워드 네트워크: 각 레이어에서 동일한 방식으로 처리되는 두 개의 선형 변환을 사용하여 입력을 변환.

- 위치 인코딩: 순차적인 관계를 학습할 수 있도록 임베딩에 사인과 코사인 함수를 사용하여 위치 정보를 추가.

4 Why Self-Attention

1. 계산 복잡도 및 병렬화:

- 자기 주의(self-attention): 한 레이어에서 모든 위치를 병렬 처리할 수 있으며, 순차적 연산이 필요하지 않아 계산이 빠릅니다. 시퀀스 길이가 표현 차원보다 짧을 때는 순환 네트워크보다 빠릅니다.
- 순환 신경망(RNN): 입력과 출력을 순차적으로 처리해야 하므로 $O(n)$ 시간 복잡도를 가집니다.
- 합성곱 신경망(CNN): $O(n/k)$ 또는 $O(\log k(n))$ 복잡도를 가집니다. 일반적으로 순환 신경망보다 비효율적입니다.

2. 긴-range 의존성:

- Self-Attention: 모든 위치가 직접 연결되므로 긴-range 의존성을 쉽게 학습할 수 있습니다.
- 순환 신경망(RNN): 긴 시퀀스에 대해 긴 경로를 따라 신호를 전파해야 하므로 긴-range 의존성 학습이 어렵습니다.
- 합성곱 신경망(CNN): 커널의 크기나 개수에 따라 연결되는 범위가 제한되어, 긴-range 의존성을 잘 학습하지 못할 수 있습니다.

3. 해석 가능성:

Self-Attention는 다양한 작업을 학습하는 여러 Attention 헤드를 통해 더 해석 가능한 모델을 제공합니다. 각 Attention 헤드는 문장의 구문적, 의미적 구조와 관련된 패턴을 학습할 수 있습니다.

5 Training

1. 훈련 데이터:

영어-독일어 데이터셋(450만 문장)과 영어-프랑스어 데이터셋(3,600만 문장)을 사용. 바이트 쌍 인코딩(BPE) 방식으로 문장을 인코딩하고, 각 문장 쌍은 약 25,000개의 소스 및 타겟 토큰으로 배치됨.

2. 하드웨어 및 훈련 시간:

8개의 NVIDIA P100 GPU로 훈련.

기본 모델은 훈련 단계당 약 0.4초, 총 100,000단계(12시간) 동안 훈련.

큰 모델은 훈련 단계당 1초, 총 300,000단계(3.5일) 동안 훈련.

3. 최적화:

Adam 최적화기 사용, 학습률은 처음 4000단계 동안 선형 증가, 그 후 제곱근 역수로 감소.

4. 정규화 기법:

잔여 드롭아웃: 각 서브 레이어 출력에 드롭아웃을 적용하여 과적합을 방지.

5. 레이블 스무딩: 학습 시 레이블을 부드럽게 하여 모델이 더 불확실하게 학습하도록 유도, BLEU 점수를 향상.

6. Results

6.1 기계 번역 (Machine Translation)

WMT 2014 영어-독일어 번역에서 **대형 Transformer 모델 (Transformer (big))**은 기존의 가장 우수한 모델(앙상블 모델 포함)을 2.0 BLEU 점수 이상 초과하며, 새로운 BLEU 점수 28.4를 기록했습니다. 이 모델은 8개의 P100 GPU에서 3.5일 동안 훈련되었습니다.

WMT 2014 영어-프랑스어 번역에서는 대형 모델이 41.0 BLEU 점수를 기록하며, 이전의 모든 단일 모델을 초과했으며, 훈련 비용은 1/4 미만입니다. 영어-프랑스어 모델은 Pdrop = 0.1을 사용했습니다.

기본 모델에서는 마지막 5개의 체크포인트를 평균 내었고, 대형 모델에서는 마지막 20개의 체크포인트를 평균 내었습니다. 실험에 따라 하이퍼파라미터가 설정되었습니다.

빔드 검색(beam search)은 빔 크기 4와 길이 패널티 $\alpha = 0.6$ 을 사용하여 실행되었으며, 추론 시 최대 출력 길이는 입력 길이 + 50으로 설정되었고, 가능한 경우 일찍 종료되었습니다.

표 2는 번역 품질과 훈련 비용을 다른 모델들과 비교하여 요약한 결과입니다. 훈련 비용은 부동소수점 연산(FLOP) 수를 사용하여 추정했습니다.

6.2 모델 변형 (Model Variations)

Transformer 모델의 다양한 구성 요소의 중요성을 평가하기 위해 기본 모델을 다양한 방식으로 변형하고 성능을 측정했습니다. 주로 영어-독일어 번역에서 실험이 이루어졌습니다.

주의 헤드 수(attention heads)와 주의 키와 값 차원(dk, dv)을 변경하여 실험을 진행했으며, 너무 많은 헤드를 사용하면 성능이 저하되는 것을 확인했습니다.

주의 키 크기 감소(dk)는 모델 품질을 저하시킨다는 사실도 확인했습니다.

드롭아웃은 과적합을 방지하는 데 매우 유용하다는 결과가 나왔습니다.

사인 함수 대신 학습된 위치 임베딩을 사용했을 때, 성능은 기본 모델과 거의 동일했습니다.

6.3 영어 구문 분석 (English Constituency Parsing)

Transformer가 다른 작업에도 잘 일반화되는지 평가하기 위해 영어 구문 분석 (Constituency Parsing) 실험을 진행했습니다. 이 작업은 출력에 강한 구조적 제약이 있고, 입력보다 상당히 긴 출력이 필요합니다.

Wall Street Journal(WSJ) 데이터셋을 사용하여 4계층 Transformer 모델을 훈련시켰습니다. 훈련 데이터 수는 약 40,000개의 문장이며, 세미-슈퍼바이저드 설정에서는 1700만 문장을 사용했습니다.

결과는 Transformer가 기존의 모든 모델을 초과하여 성능이 매우 우수함을 보여주었으며, 특히 BerkeleyParser를 초과했습니다. 세미-슈퍼바이저드 설정에서 92.7 F1 점수를 기록하며, 기존의 RNN 모델을 초과하는 성과를 보였습니다.

7. Conclusion

Transformer는 주의를 기반으로 한 효율적이고 빠른 학습을 제공하며, 번역 작업에서 새로운 최고 성능을 기록했습니다. 향후 다른 작업에의 확장과 효율적인 대규모 데이터 처리를 목표로 연구를 계속할 계획입니다.